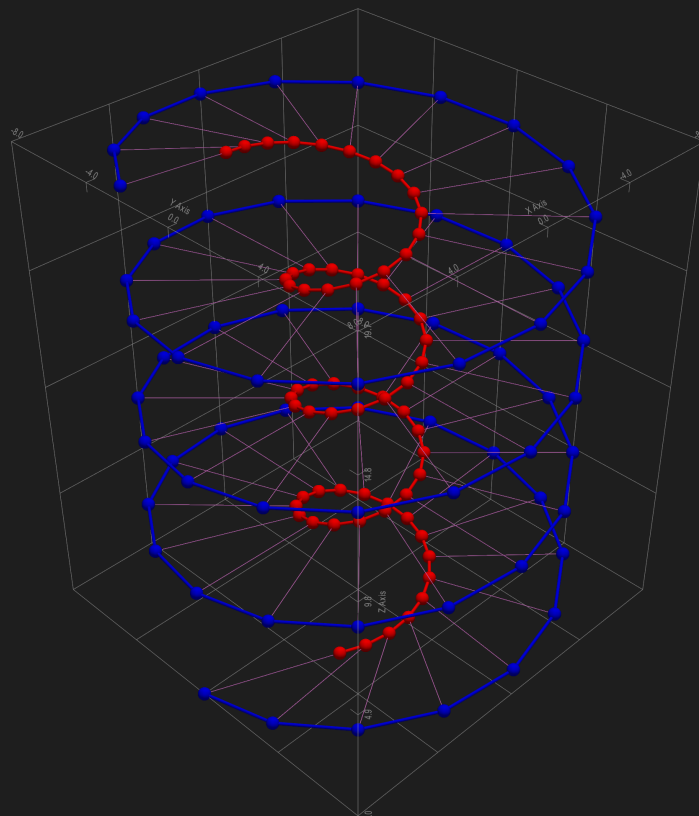


Projecting chains of distance constrained points

Morten Silcowitz

March 6, 2026



1 Introduction

This note documents a practical algorithm for projecting a chain of points with unit-length constraints on consecutive difference vectors. The goal is not to present a formal paper, but to capture the derivation and the key numerical ideas behind an experimental solver. We reformulate the problem in terms of difference vectors, derive a Newton-like system, and then exploit the resulting block tridiagonal structure for fast direct solves. The emphasis is on clear steps and implementation-friendly formulas rather than theoretical optimality guarantees.

2 Formulation

We're going to solve

$$\begin{aligned} \min_{\mathbf{x}} \quad & \frac{1}{2} (\mathbf{x} - \mathbf{p})^\top \mathbf{M} (\mathbf{x} - \mathbf{p}) \\ \text{s.t} \quad & \mathbf{z} = \mathbf{R}\mathbf{x} \quad \text{and} \quad \mathbf{z}_i^\top \mathbf{z}_i = 1 \end{aligned} \tag{1}$$

where

$n \in \mathbb{N}$	is the number of particles
$m \in \mathbb{N}$	is the number of links, $m = n - 1$
$\mathbf{x} \in \mathbb{R}^{3n}$	is the solution vector,
$\mathbf{p} \in \mathbb{R}^{3n}$	is target/free moving positions,
$\mathbf{z} \in \mathbb{R}^{3m}$	is the vector of difference vectors in the chain,
$\mathbf{z}_i \in \mathbb{R}^3$	is the i th 3-vector in $\mathbf{z}$,
$\mathbf{R} \in \mathbb{R}^{3m \times 3n}$	is the per-edge finite-difference matrix so that $(\mathbf{R}\mathbf{x})_i = \mathbf{x}_i - \mathbf{x}_{i+1}$
$\mathbf{M} \in \mathbb{R}^{3n \times 3n}$	is the mass matrix, simply having the mass (or weight) of each particle in each 3×3 block diagonal

We want to be working only on the difference vectors in $\mathbf{z}$. To do that, we need to eliminate $\mathbf{x}$, and we do this by holding $\mathbf{z}$ constant and deriving the optimality conditions for (1). We first state the Lagrangian:

$$\mathcal{L}(\mathbf{x}, \lambda) = \frac{1}{2}(\mathbf{x} - \mathbf{p})^\top \mathbf{M}(\mathbf{x} - \mathbf{p}) + \lambda^\top (\mathbf{z} - \mathbf{R}\mathbf{x})$$

with the optimality conditions:

$$\begin{aligned} \mathbf{M}(\mathbf{x} - \mathbf{p}) - \mathbf{R}^\top \lambda &= 0 \\ \mathbf{z} - \mathbf{R}\mathbf{x} &= 0 \end{aligned}$$

solving for λ gives us:

$$\lambda = (\underbrace{\mathbf{R}\mathbf{M}^{-1}\mathbf{R}^\top}_S)^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p})$$

where $\mathbf{S} = \mathbf{R}\mathbf{M}^{-1}\mathbf{R}^\top \in \mathbb{R}^{3m \times 3m}$ is a block tridiagonal SPD matrix. We use this to give us an expression for $\mathbf{x}$:

$$\mathbf{x} = \mathbf{p} + \mathbf{M}^{-1}\mathbf{R}^\top \mathbf{S}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p})$$

Inserting this into (1) eliminates $\mathbf{z} = \mathbf{R}\mathbf{x}$ and we are left with

$$\begin{aligned} \min_{\mathbf{z}} \quad & \frac{1}{2} (\mathbf{z} - \mathbf{R}\mathbf{p})^\top \mathbf{S}^{-1} (\mathbf{z} - \mathbf{R}\mathbf{p}) \\ \text{s.t.} \quad & \mathbf{z}_i^\top \mathbf{z}_i = 1 \quad i = 1 \dots m \end{aligned} \quad (2)$$

In this reformulated problem we must find a $\mathbf{z}$ that consists of unit length 3-vectors that minimize the distance to $\mathbf{R}\mathbf{p}$ under the $\mathbf{S}^{-1}$ -norm. Note that it is trivial to project $\mathbf{z}$ to satisfy the unit length constraints (under the regular $\mathbf{I}$ -norm, known as normalization), and likewise it is trivial to obtain $\mathbf{x}$ given any $\mathbf{z}$. This means that whenever we take a Newton step on $\mathbf{z}$ we can trivially rein it to its feasible solution, and in effect only change the directions of each $\mathbf{z}_i$ vector, never their lengths. Throughout, $\mathbf{M}$, $\mathbf{R}$, $\mathbf{S}$, and $\mathbf{p}$ are fixed inputs; $\mathbf{z}$ is the variable we iterate, and λ is re-estimated each loop.

3 Solution

We want to solve this system using a Newton-like method, so to make sense of that we first derive the full Newton step. The Lagrangian of (2) is

$$\mathcal{L}_{(\mathbf{z}, \lambda)} = \frac{1}{2} (\mathbf{z} - \mathbf{R}\mathbf{p})^\top \mathbf{S}^{-1} (\mathbf{z} - \mathbf{R}\mathbf{p}) + \frac{1}{2} \sum_{i=1}^m \lambda_i (\mathbf{z}_i^\top \mathbf{z}_i - 1) \quad (3)$$

The optimality conditions for (3) are:

$$\begin{aligned} \mathbf{S}^{-1} (\mathbf{z} - \mathbf{R}\mathbf{p}) + \mathbf{Q}_z^\top \lambda &= 0 \\ \mathbf{z}_i^\top \mathbf{z}_i - 1 &= 0 \quad i = 1, \dots, m \end{aligned}$$

and the Newton step to find a stationary point $(\mathbf{z}, \lambda)$ for (3) is

$$\underbrace{\begin{bmatrix} \mathbf{S}^{-1} + \mathbf{D}_\lambda & \mathbf{Q}_z^\top \\ \mathbf{Q}_z & \mathbf{0} \end{bmatrix}}_{H\mathcal{L}_{(\mathbf{z}, \lambda)}} \begin{bmatrix} \Delta \mathbf{z} \\ \Delta \lambda \end{bmatrix} = - \underbrace{\begin{bmatrix} \mathbf{S}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}) + \mathbf{Q}_z^\top \lambda \\ \frac{1}{2}(\mathbf{Q}_z \mathbf{z} - \mathbf{1}) \end{bmatrix}}_{\nabla \mathcal{L}_{(\mathbf{z}, \lambda)}} \quad (4)$$

where

$$\mathbf{Q}_z = \begin{bmatrix} \mathbf{z}_1^\top & 0 & \cdots & 0 \\ 0 & \mathbf{z}_2^\top & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{z}_m^\top \end{bmatrix} \in \mathbb{R}^{m \times 3m} \quad \mathbf{D}_\lambda = \begin{bmatrix} \mathbf{I}_{\lambda_1} & 0 & \cdots & 0 \\ 0 & \mathbf{I}_{\lambda_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{I}_{\lambda_m} \end{bmatrix} \in \mathbb{R}^{3m \times 3m}$$

There are many interesting ways to go about solving this system, but in our case the goal is to exploit the tridiagonal structure of $\mathbf{S}$, in order to use fast direct solves. Instead of solving the full step, we first solve just for $\Delta \lambda$ at $\lambda = \mathbf{0}$, effectively resetting the multipliers in each iteration. This is not standard Newton, but in practice it improves stability and keeps each linear system sparse and well-conditioned. After that we solve for $\Delta \mathbf{z}$ to update the solution variables.

3.1 Solving for λ

To derive the solution for $\Delta \lambda$ we eliminate $\Delta \mathbf{z}$ by using $\lambda = \mathbf{0}$ and solving the first row in (4). Interestingly, because we will project the vectors in $\mathbf{z}$ in each iteration, the $\mathbf{Q}_z \mathbf{z} - \mathbf{1}$ term is always 0, which will simplify things a bit:

$$\Delta \mathbf{z} = -\mathbf{S}(\mathbf{Q}_z^\top \Delta \lambda + \mathbf{S}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}))$$

Substitute $\Delta \mathbf{z}$ into the second row in (4):

$$\mathbf{Q}_z(-\mathbf{S}(\mathbf{Q}_z^T \Delta \lambda + \mathbf{S}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}))) = \frac{1}{2} \underbrace{(\mathbf{Q}_z \mathbf{z} - \mathbf{1})}_0$$

Simplifying to solve for $\Delta \lambda$, we get:

$$\mathbf{Q}_z \mathbf{S} \mathbf{Q}_z^T \Delta \lambda = -\mathbf{Q}_z(\mathbf{z} - \mathbf{R}\mathbf{p})$$

This system is tridiagonal because the operator $\mathbf{Q}_z(\cdot)\mathbf{Q}_z^T$ collapses each 3×3 block in $\mathbf{S}$ to a scalar value $\mathbf{z}_i^T \mathbf{S}_{(i,j)} \mathbf{z}_j$. This operation effectively reduces the block tridiagonal structure of $\mathbf{S}$ to the plain tridiagonal structure in $\mathbf{Q}_z \mathbf{S} \mathbf{Q}_z^T \in \mathbb{R}^{m \times m}$. Also note that $\mathbf{Q}_z \mathbf{z} = \mathbf{1}$ appears in the RHS and so it could be further reduced to $\mathbf{Q}_z \mathbf{R}\mathbf{p} - \mathbf{1}$ but since the vector $\mathbf{z} - \mathbf{R}\mathbf{p}$ is needed elsewhere this has negligible practical benefit.

3.2 Solving for $\Delta \mathbf{z}$

To solve for $\Delta \mathbf{z}$, we substitute $\Delta \lambda = 0$ into the first row of (4):

$$(\mathbf{S}^{-1} + \mathbf{D}_\lambda) \Delta \mathbf{z} = -\mathbf{S}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}) - \mathbf{Q}_z^T \lambda \quad (5)$$

To avoid having to solve the dense system $\mathbf{S}^{-1} + \mathbf{D}_\lambda$ we exploit that we can cheaply compute the cholesky decomposition $\mathbf{L}\mathbf{L}^T = \mathbf{S}$ and make use of the identity

$$\mathbf{S}^{-1} + \mathbf{D}_\lambda = \mathbf{L}^{-T}(\mathbf{I} + \mathbf{L}^T \mathbf{D}_\lambda \mathbf{L}) \mathbf{L}^{-1} \quad (6)$$

We insert (6) into (5)

$$\mathbf{L}^{-T}(\mathbf{I} + \mathbf{L}^T \mathbf{D}_\lambda \mathbf{L}) \mathbf{L}^{-1} \Delta \mathbf{z} = -\mathbf{L}^{-T} \mathbf{L}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}) - \mathbf{Q}_z^T \lambda$$

Then, multiplying with the invertible $\mathbf{L}^T$

$$(\mathbf{I} + \mathbf{L}^T \mathbf{D}_\lambda \mathbf{L}) \mathbf{L}^{-1} \Delta \mathbf{z} = -\mathbf{L}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}) - \mathbf{L}^T \mathbf{Q}_z^T \lambda$$

means we can solve for $\mathbf{u} = \mathbf{L}^{-1} \Delta \mathbf{z}$ and then obtain $\Delta \mathbf{z}$ like so

$$(\mathbf{I} + \mathbf{L}^T \mathbf{D}_\lambda \mathbf{L}) \mathbf{u} = -\mathbf{L}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p}) - \mathbf{L}^T \mathbf{Q}_z^T \lambda$$

and then finally

$$\Delta \mathbf{z} = \mathbf{L} \mathbf{u}$$

The great advantage here is that $(\mathbf{I} + \mathbf{L}^T \mathbf{D}_\lambda \mathbf{L})$ remains block tridiagonal and so it's fast to solve. The disadvantage is that we need one extra solve of the block bidiagonal $\mathbf{L}$ to compute $\mathbf{L}^{-1}(\mathbf{z} - \mathbf{R}\mathbf{p})$. This is annoying but still greatly outweighs the pain of solving a dense system using iterative methods. Note also that in the trivial case where $\lambda = \mathbf{0}$ the system reduces to the identity. In an effort to keep the eigenvalues of $(\mathbf{I} + \mathbf{L}^T \mathbf{D}_\lambda \mathbf{L})$ positive, we clamp λ to positive values before we compute $\mathbf{D}_\lambda$. This is a hack but seems to work well in practice. We now proceed to list the full algorithm.

4 Algorithm

Here we'll outline the algorithm in full before explaining each step in detail:

```
01  $\mathbf{L} = \text{cholesky}(\mathbf{S})$ 
02  $\mathbf{z} = \mathbf{R}\mathbf{p}$  // or warm start from previous  $\mathbf{z}$ 
03 while true do
04      $\mathbf{Q}, \mathbf{z} = \text{normalize}(\mathbf{z})$ 
05      $\mathbf{y} = \mathbf{R}\mathbf{p} - \mathbf{z}$ 
06      $\lambda = \text{solve}(\mathbf{Q}\mathbf{S}\mathbf{Q}^T, \mathbf{Q}\mathbf{y})$ 
07      $\mathbf{b}_z = \text{solve}(\mathbf{L}, \mathbf{y})$ 
08      $\mathbf{b}_\lambda = \mathbf{L}^T \mathbf{Q}^T \lambda$ 
09     if  $\frac{\|\mathbf{b}_z - \mathbf{b}_\lambda\|}{\|\mathbf{R}\mathbf{p}\|} < \epsilon_{tol}$  then
10          $\mathbf{s} = \text{solve}(\mathbf{L}^T, \mathbf{b}_z)$ 
11          $\mathbf{x} = \mathbf{p} - \mathbf{M}^{-1} \mathbf{R}^T \mathbf{s}$ 
12         return  $\mathbf{x}$ 
13      $\mathbf{D} = \text{diagonal}(\max(0, \lambda))$ 
14      $\Delta \mathbf{z} = \mathbf{L} \text{ solve}(\mathbf{I} + \mathbf{L}^T \mathbf{D} \mathbf{L}, \mathbf{b}_z - \mathbf{b}_\lambda)$ 
15      $\mathbf{z} = \mathbf{z} + \Delta \mathbf{z}$ 
```

Each iteration the algorithm does these main steps:

1. Normalize the 3-vectors in $\mathbf{z}$
2. Compute the $n - 1$ Lagrange multipliers of the system at $\mathbf{z}$
3. Check the solution residual, if below threshold compute and return $\mathbf{x}$
4. Compute the Newton step on $\mathbf{z}$ and go to step 1

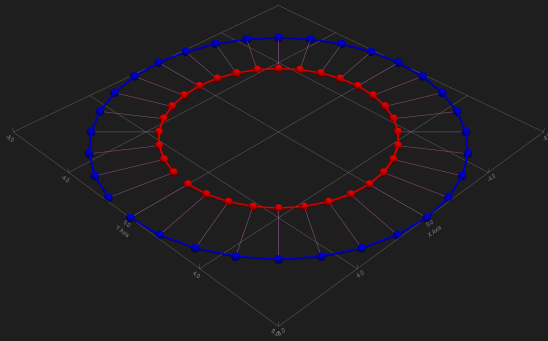
Note the following:

- we precompute the Cholesky factor $\mathbf{L}\mathbf{L}^T = \mathbf{S}$ once if $\mathbf{M}$ is fixed, and reuse it in every iteration to keep all solves sparse
- the sign convention is handled by setting $\mathbf{y} = \mathbf{R}\mathbf{p} - \mathbf{z}$, so the Newton right-hand side becomes $\mathbf{b}_z - \mathbf{b}_\lambda$ in the factored system
- $\mathbf{Q}$ is block diagonal with row blocks $\mathbf{z}_i^T$, so $\mathbf{Q}\mathbf{S}\mathbf{Q}^T$ is plain tridiagonal and fast to solve
- clamping λ before forming $\mathbf{D}$ keeps $\mathbf{I} + \mathbf{L}^T\mathbf{D}\mathbf{L}$ positive definite; this is a pragmatic stabilizer rather than a theoretical guarantee

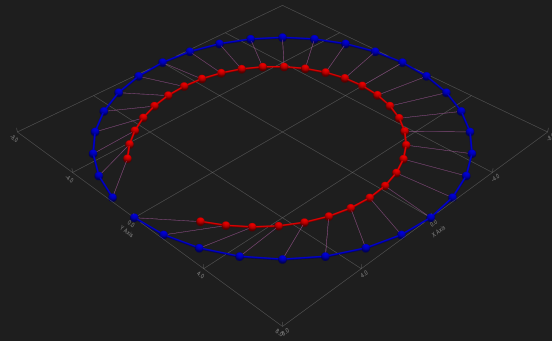
5 Test

We find that the best way to get a feel for how this algorithm behaves in animation and/or simulation settings, is to play around with the interactive demo. The C implementation used in the following tests as well as in the demo is a naive implementation that just operates on the raw input. There is no scaling, clamping, or other techniques to avoid blow-ups or errors. Notably there is no line-search type handling of the step-size, the solution is simply normalized per $\mathbf{z}_i$ vector after each update. As is probably expected, the algorithm converges more slowly when the difference between the initial starting solution and the final solution is large. Often the initial solution is close to the converged solution in animation problem. Still the algorithm shows decent linear convergence to machine precision in the following tests, where the final solution is far from the initial and also far from the target positions. We find that in animation type scenarios, convergence can be much faster.

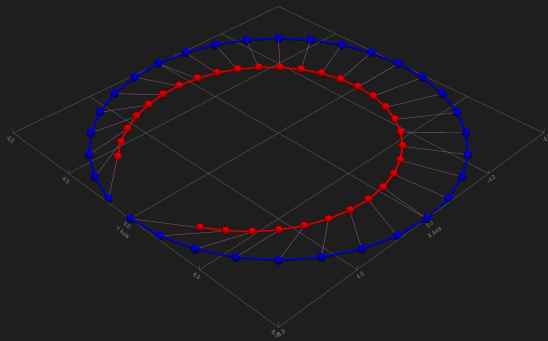
Circle



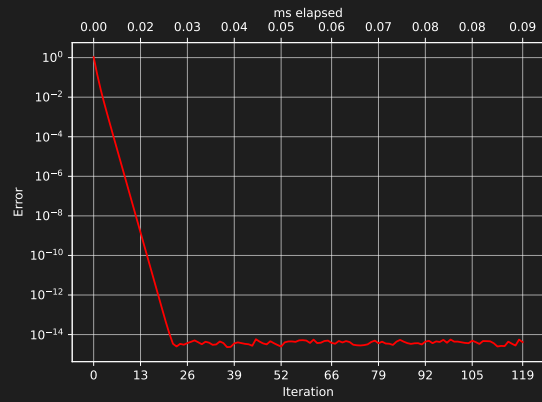
(a) Iteration 0



(b) Iteration 1



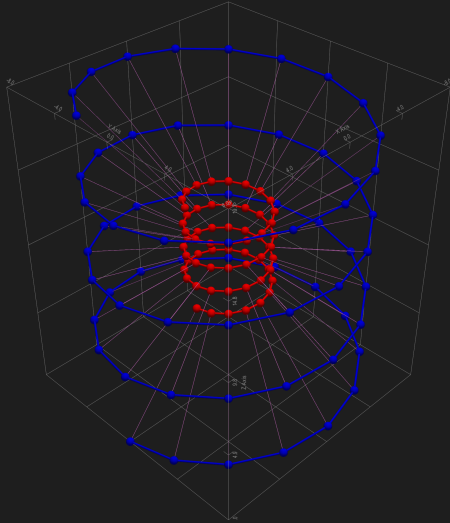
(c) Last iteration



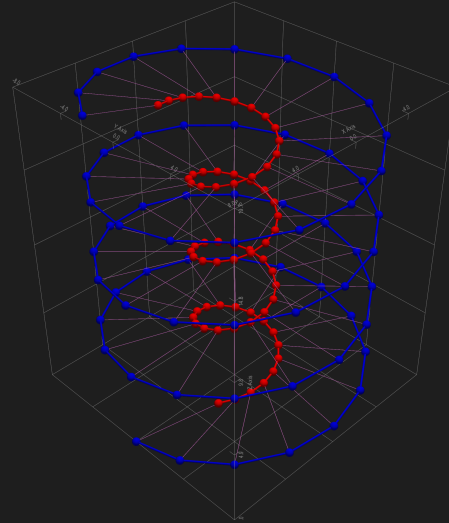
(d) Convergence

Figure 1: The blue curve shows the target positions, while the red curve shows the solution positions. The points are rendered as a continuous curves for better visualization. $N=32$, finished in 21 iterations at $e=4.45e-14$ in 0.0247 ms running on 13th Gen Intel(R) Core(TM) i9-13900KS

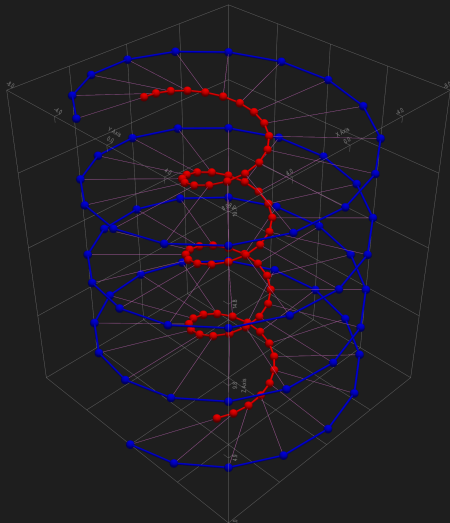
Helix



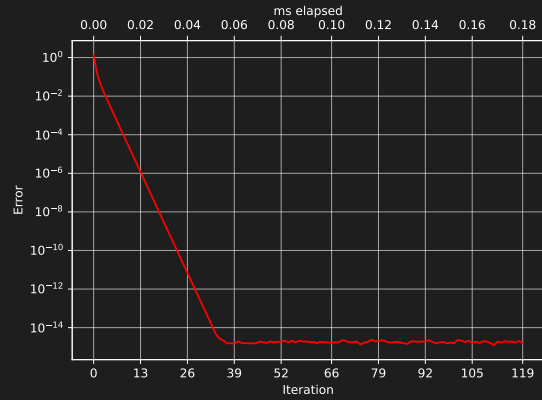
(a) Iteration 0



(b) Iteration 1



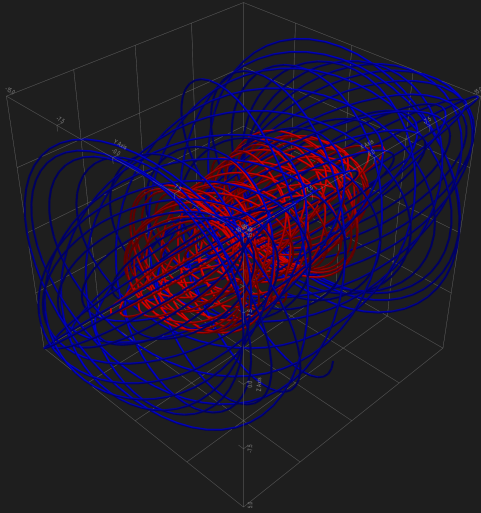
(c) Last iteration



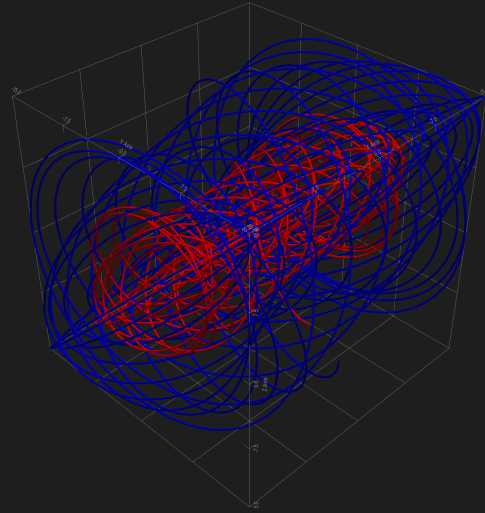
(d) Convergence

Figure 2: The blue curve shows the target positions, while the red curve shows the solution positions. The points are rendered as a continuous curves for better visualization. $N=64$, finished in 32 iterations at $e=6.76e-14$ in 0.0491 ms running on 13th Gen Intel(R) Core(TM) i9-13900KS

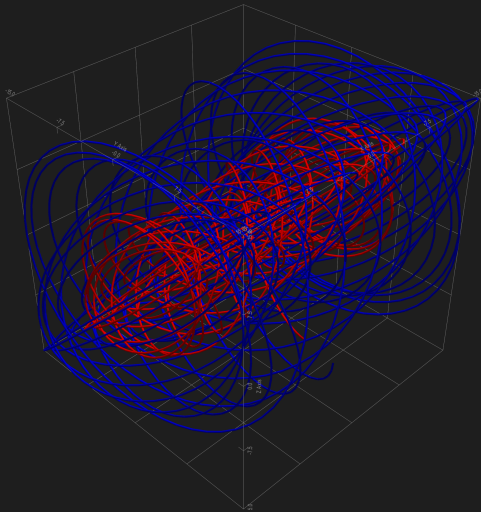
Knot



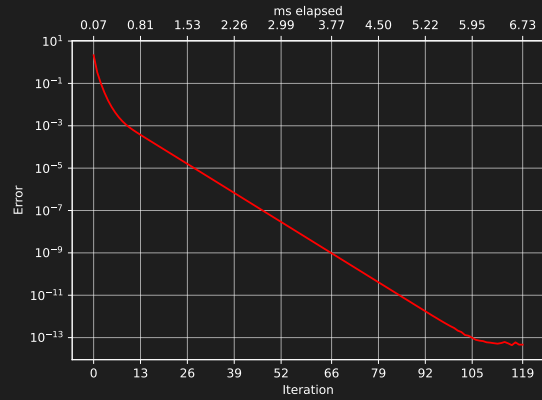
(a) Iteration 0



(b) Iteration 1



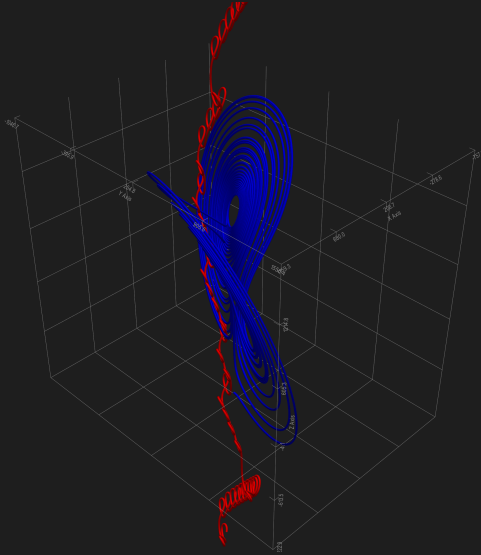
(c) Last iteration



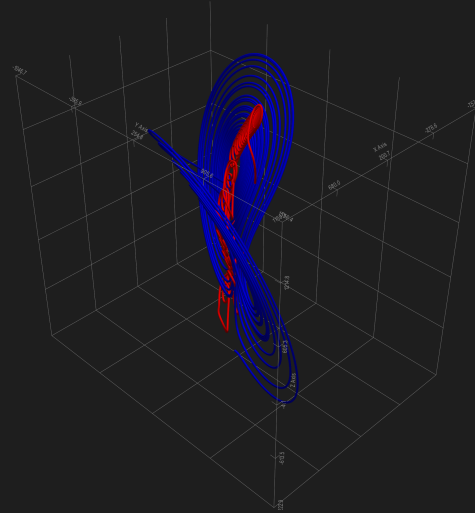
(d) Convergence

Figure 3: The blue curve shows the target positions, while the red curve shows the solution positions. The points are rendered as a continuous curves for better visualization. $N=2048$, finished in 106 iterations at $e=9.82e-14$ in 5.9480 ms running on 13th Gen Intel(R) Core(TM) i9-13900KS

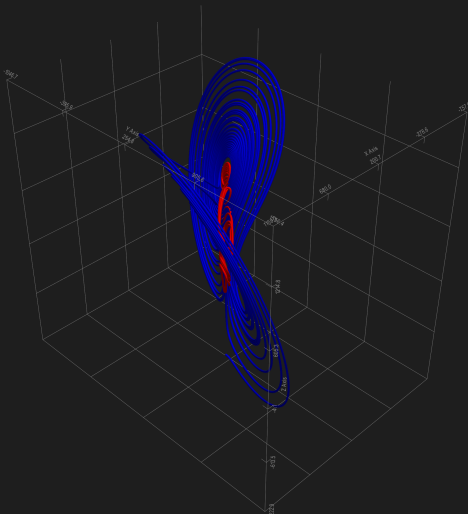
Lorenz



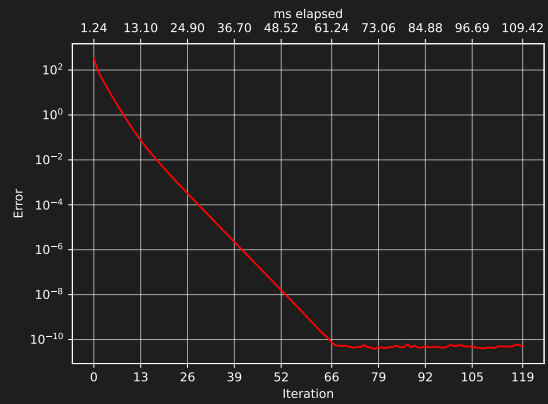
(a) Iteration 0



(b) Iteration 1



(c) Last iteration



(d) Convergence

Figure 4: The blue curve shows the target positions, while the red curve shows the solution positions. The points are rendered as a continuous curves for better visualization. $N=32768$, finished in 79 iterations at $e=3.77e-11$ in 72.1546 ms running on 13th Gen Intel(R) Core(TM) i9-13900KS